

Applications Development and Emerging Technologies

MODULE 1

Web Architecture, Tools, and Introduction to PHP



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Objectives

- To understand the history of PHP
- To know the advantages of PHP as a server side scripting language
- To know what are the software's needed in developing web application using PHP.
- To run PHP application on a web browser.
- To know the basic syntax of PHP for outputting to browser.
- To know the datatypes that are available on PHP and on how these datatypes are being used.
- To know some predefined type function for data manipulation.
- To know the proper casting values of a given variables.



HISTORY

- **1994/1995 (PHP 1.0)**
 - *Developed by Rasmus Lerdorf*
 - *To know how many visitors were reading his online resume based on PERL/CGI script*
 - *Personal Home Page (PHP)*
- **1997 (PHP 2.0)**
 - *PHP is based on C rather than PERL*
 - *Personal Home Page/Form Interpreter*
- **1998 (PHP 3.0)**
 - *50,000 users were using PHP to enhance their Web pages*
 - *Developers joined Lerdorf*
- **1999 (PHP 4.0)**
 - *With core developers Zeev Suraski and Andi Gutmans*
 - *PHP makes the most popular scripting language with morethan one million user base by Netcraft*



HISTORY (Con't.)

- *Hundreds of functions being added*
- *Dubbed the Zend scripting engine*
- *May 22, 2000 (PHP 4.0)*
 - *PHP: Hypertext Preprocessor (recursive acronym)*
 - *3.6 million domain PHP installed*
 - *Enterprise development*
 - *Improved resource handling (scalability)*
 - *Object-oriented support (Classes and Objects)*
 - *Native session-handling support (session)*
 - *Encryption (encryption algorithms)*
 - *ISAPI Support (for IIS)*
 - *Native COM/DOM (Windows applications)*
 - *Native Java Support: (binding to java objects)*
 - *PERL Compatible Regular Expressions*
- *New features, power, and scalability*



HISTORY (Con't.)

- *PHP 5 (July 13, 2004)*
 - *Vastly improved object-oriented capabilities: OOP improvement*
 - *Try/catch exception handling*
 - *Improved XML and Web Services support (Simple XML Support, using SOAP)*
 - *Native support for SQLite*
 - *Installed on 19 million domains*
 - *54 percent on all Apache module*
- *PHP 6 (2007 not yet released)*
 - *Unicode Support (Native Unicode support for multilingual applications)*
 - *Security improvements*
 - *New language features and constructs (64-bit type integer, foreach looping)*

<http://en.wikipedia.org/wiki/PHP>



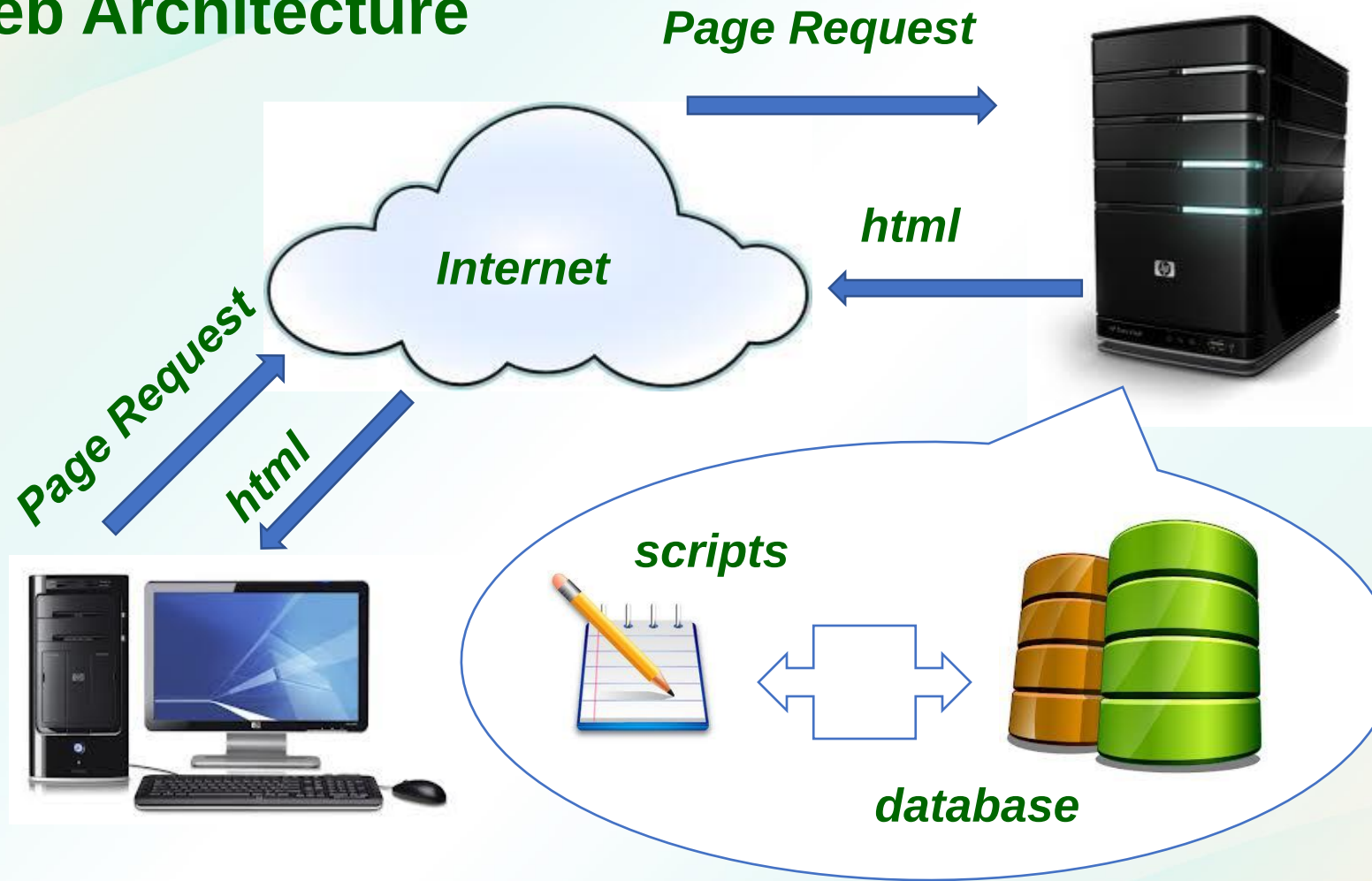
HISTORY (Con't.)

PHP Key categories

- *Practicality*
 - *PHP is a loosely type language (no explicitly create, typecast, or destroy a variable)*
- *Power*
 - *More libraries and thousands of functions*
- *Possibility*
 - *Native support is offered for more than 25 database products, including Adabas D, dBase, Empress, FilePro, FrontBase, Hyperwave, IBM DB2, Informix, Ingres, InterBase, mSQL, Microsoft SQL Server, MySQL, Oracle, Ovrimos, PostgreSQL, Solid, Sybase, Unix dbm, and Velocis.*
 - *Both structured and Object Oriented approach*
- *Price*
 - *Free of charge*

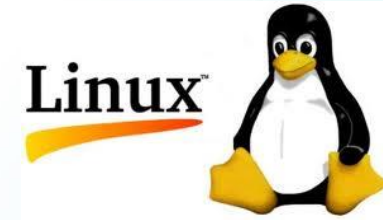


Web Architecture



Software Requirements

Operating System: Linux



Technology Driven by Innovation

Software Requirements

Operating System: Windows



Software Requirements

Operating System: Mac



Cheetah

Panther

Puma

Jaguar

Tiger



Leopard

Snow leopard

Lion

Software Requirements

Internet browser



**Google
Chrome**



**Mozilla
Firefox**



**Internet
Explorer**



Opera



Safari



Maxthon



Rockmelt



SeaMonkey



**Deepnet
Explorer**



**Avant
Browser**

Software Requirements Web Server



Apache



Google Web Server (GWS)



Software Requirements

Server side scripting language



*Java
(JavaServer Pages: JSP)*



Ruby (Ruby on Rails)



*Active Server Pages
(ASP)*



*Python
(Django)*



PERL CGI



Software Requirements Database



Software Requirements

Code editor



Bluefish



Dreamweaver



NetBeans



Notepad++

Software Requirements

Other

HTML

CSS



Software Requirements

Other



Operating System
Windows 7 / Windows 8



Web Server
Apache



Language Script
PHP



Database
MySQL



XAMPP

XAMPP / WAMP



WampServer



Code Editor

Notepad++ / Eclipse / Netbeans / Dreamweaver



FEU ALABANG



FEU DILIMAN

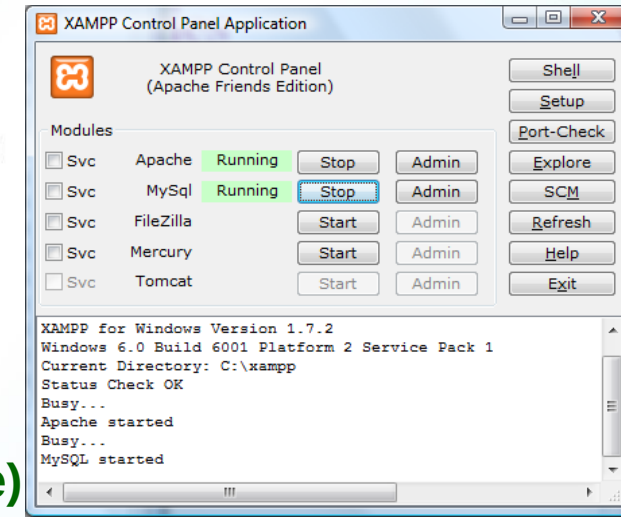


FEU TECH

Running PHP Scripts XAMPP



1. Download XAMPP
<http://www.apachefriends.org/en/xampp.html>
2. Install XAMPP
3. Run XAMPP Control
 - c:\xampp (by default)
 - Start the Apache and MySQL(for database)
4. Create a folder under c:\xampp\htdocs\ (by default)
Note: All php files must be save on that folder
5. Test PHP script sample file.
 5. Open some internet browser
 6. Type localhost/[folder name] (by default)
 7. Select file from the directory list (if there's



Running PHP Scripts

```
<!-- welcome.php -->
<html>
<head>
<title>My PHP</title>
</head>
<body>
```

```
<?php
    echo 'Welcome to PHP';
?>
```

```
</body>
</html>
```

All php scripts must enclosed with

<?php ... ?> (standard tag)

<? ... ?> (short open tag)

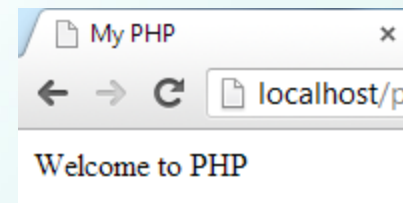
<script language="php">

....

</script> (script tag)

<% ... %> (asp style tag)

OUTPUT:



Running PHP Scripts

opening tag for php script

```
<?php
```

Statement terminator

```
echo 'Welcome to PHP';
```

```
?>
```

closing tag for php script

echo – is a language construct that use to output one or more string.

print – is a function also use for outputting strings.

PHP Comments

```
<?php
    // single line comment (C syntax)
    # single line comment (PERL style)
    /*
        multiline
        or block
        comments
    */
?>
```

PHP Variables

- Variables are used as a container for values that can be used and manipulate PHP scripts.
- Variables in PHP must begin with a \$ symbol.
- Variable might contain a string, numbers, arrays and objects.
- Setting the identifier for a variable must follow some rules.
 1. First character can be '_' or a letter.
 2. It can only contain alpha-numeric characters and underscore (a-z,A-Z,0-9,_).
 3. Don't use white space in naming a variable.
- Variables in PHP is not explicitly declared
- Variables are case sensitive

Example

```
$age, $firstName, $totalSalary, $_myValue,  
$str1, $tmp01
```



Using ‘ ‘ (pair single quote), “ “ (pair double quotes) and . (dot character) PHP

1. Strings inside the ‘ and ‘ are interpreted as literal strings
 1. To display a ‘ character (single quote) as string use the escape character \ (backslash).
2. Strings inside the “ and “ are interpreted as literal strings with some exceptions
 1. \$ inside “ and “ are interpreted as variables thus it will display the value rather than the string that starts with \$.
 2. To display the string that starts with \$ and to display the double quote as string use the escape character \ (backslash).
3. To concatenate string values use the . (dot or period) character thus “abc“.”def” yields to a value “abcdef”

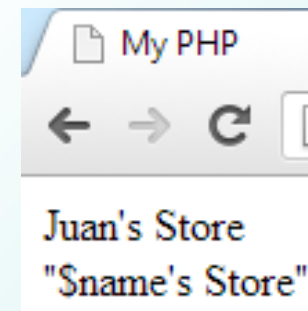


Using ' ' (pair single quote), " " (pair double quotes) and . (dot character) PHP

Example:

```
<?php
    $name = "Juan";
    echo "$name's Store"."<br/>";
    echo '"$name\'s Store"'. "<br/>";
    //statements below will produced errors
    //echo "displaying double quote" ";
    //echo "dispalying single quote ' ";
?>
```

Output:



Juan's Store
"\$name's Store"

Outputting Data

echo syntax

```
void echo(string argument1[,...string argumentN]
```

print syntax

```
int print(argument)
```

printf syntax

```
boolean printf(string format [, mixed args])
```

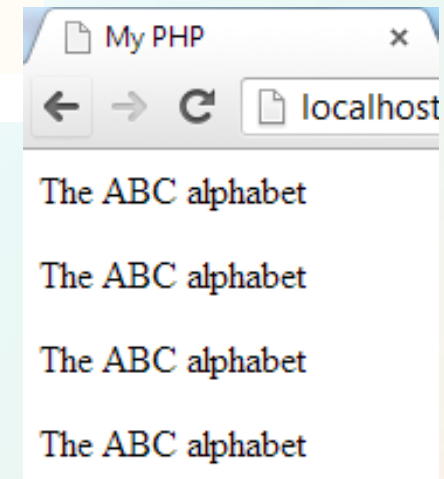


Outputting Data

Example:

```
<?php
//using echo
echo "<p>The ABC alphabet</p>";
echo "<p>","The ","ABC ","alphabet","</p>";
//using print statement
print("<p>The ABC alphabet</p>");
//using printf statement
printf("<p>%s</p>","The ABC alphabet");
?>
```

Output:



An introduction to PHP web programming

Commonly Used Type Specifiers for printf function

Type	Description
%b	Argument considered an integer; presented as a binary number
%c	Argument considered an integer; presented as a character corresponding to that ASCII value
%d	Argument considered an integer; presented as a signed decimal number
%f	Argument considered a floating-point number; presented as a floating-point number
%o	Argument considered an integer; presented as an octal number
%s	Argument considered a string; presented as a string
%u	Argument considered an integer; presented as an unsigned decimal number
%x	Argument considered an integer; presented as a lowercase hexadecimal number
%X	Argument considered an integer; presented as an uppercase hexadecimal number

Beginning PHP and MySQL 3rd Edition by: W. Jason Gilmore pp.100



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

An introduction to PHP web programming

PHP Datatypes

Datatypes

is the generic name assigned to any data sharing a common set of characteristics.

Scalar Datatypes

Capable of containing a single item of information
(Boolean, Integer, Float (float, double or real numbers), String)

Compound Datatypes

allow for multiple items of the same type to be aggregated under a single representative entity.
(Array and Objects)



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

An introduction to PHP web programming

Scalar Datatypes Example:

```
//Integer
$a = 13;           //13
$a = -12345;       //-12345
$a = 012;          //10
$a = 0x1A;         //26
```

```
//float
$v = 3.2475;       //3.2475
$v = 2.0;          //2
$v = 8.7e4;        //87000
$v = 1.43E+10;     //143E+10
```

```
//string
$str = "Welcome to PHP"; //Welcome to PHP
$str = "do-re-mi";       //do-re-mi
$str = '$^expr%!';      //$^expr%!
$str = "123$%^789";     //123$%^789
```

```
//Boolean
$a = true;          //$a is true
$a = 1;             //$a is true
$a = -1;            //$a is true
$a = 1056;          //$a is true
$a = 0;             //$a is false
$a = false;         //$a is false
```



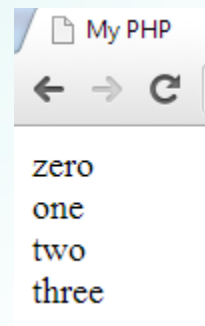
An introduction to PHP web programming

Compound Datatypes

Example:

```
<?php
//arrays
$arr_list[0] = "zero";
$arr_list[1] = "one";
$arr_list[2] = "two";
$arr_list[3] = "three";
for($i=0;$i<count($arr_list);$i++)
    echo $arr_list[$i]."<br/>";
?>
```

Output:

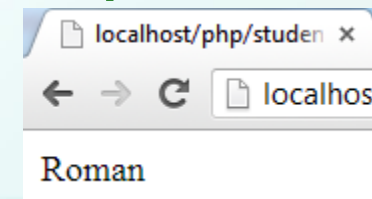


zero
one
two
three

```
<?php
//objects
class Student{
    private $_name;
    function setName($name){
        $this->_name = $name;
    }
    function getName(){
        return $this->_name;
    }
}

$stud = new Student;
$stud->setName('Roman');
echo $stud->getName();
?>
```

Output:



Roman



An introduction to PHP web programming

PHP Type Casting Operators

Cast Operators	Conversion
(array)	Array
(bool) or (boolean)	Boolean
(int) or (integer)	Integer
(int64)	64-bit integer (introduced in PHP 6)
(object)	Object
(real) or (double) or (float)	Float
(string)	String

Beginning PHP and MySQL 3rd Edition by: W. Jason Gilmore pp.105



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

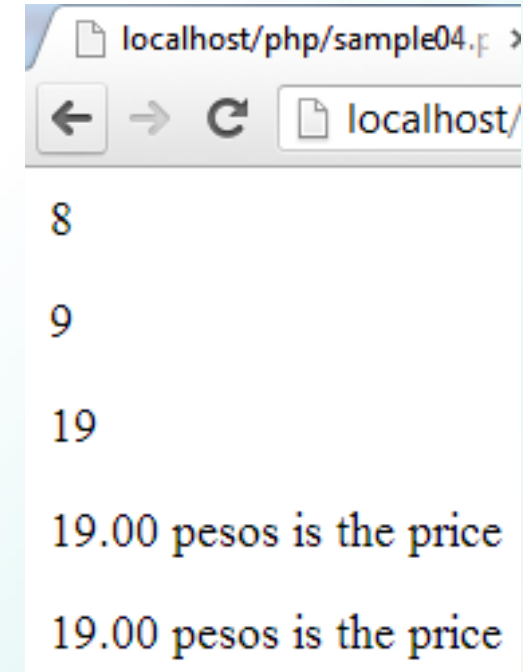
An introduction to PHP web programming

PHP Type Casting Operators

Example:

```
<?php
    $x = (double) 8;
    echo "<p>$x</p>";
    $x = (int) 9.99;
    echo "<p>$x</p>";
    $str = "19.00 pesos is the price";
    $x = (int) $str;
    echo "<p>$x</p>";
    $x = (array) $str;
    echo "<p>$x[0]</p>";
    $obj = (object) $str;
    echo "<p>$obj->scalar</p>";
?>
```

Output:



localhost/php/sample04.php

8

9

19

19.00 pesos is the price

19.00 pesos is the price



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

An introduction to PHP web programming

PHP Type Juggling

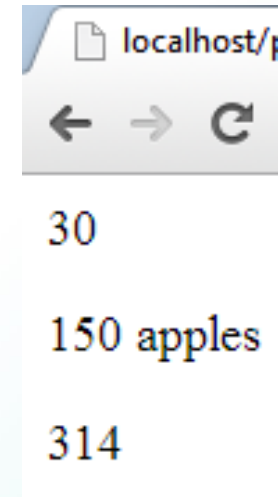
Example:

```
<?php
    $a = 10;
    $b = "20";
    $c = $a + $b;
    echo "<p>$c</p>";

    $a = "100 apples";
    $b = "50 apples";
    $c = $a + $b;
    echo "<p>$c apples</p>";

    $a = "314e-2";
    $b = 100;
    $c = $a * $b;
    echo "<p>$c</p>";
?>
```

Output:



An introduction to PHP web programming

Predefined Type Functions

`gettype()` function

returns the type of the variable. Possible return values are integer, double, boolean, string, array, object, resource, unknown types.

prototype:

```
string gettype(mixed var)
```

`settype()` function

converts a given variable to a specific type.

prototype:

```
boolean settype(mixed var, string type)
```

Predefined Type Identifier Functions

```
is_array(), is_bool(), is_float(), is_integer(),  
is_null(), is_numeric(), is_object(), is_resource(),  
is_scalar(), and is_string()
```



An introduction to PHP web programming

Type Functions

Example:

```
<?php
    $x = 10;
    printf("Variable type: %s <br>", gettype($x));
    settype($x, double);
    printf("Variable type: %s <br>", gettype($x));

    if(is_integer($x)){
        echo "Integer: True<br/>";
    } else {
        echo "Integer: False<br/>";
    }

    if(is_double($x)){
        echo "Double: True<br/>";
    } else {
        echo "Double: False<br/>";
    }

    if(is_numeric($x)){
        echo "Numeric: True<br/>";
    } else {
        echo "Numeric: False<br/>";
    }

?>
```

Output:

```
Variable type: integer
Variable type: double
Integer: False
Double: True
Numeric: True
```



Summary

- *PHP was developed originally by Rasmus Lerdorf*
- *Seev Suraski and Andi Gutsman were the one who contribute most in PHP development.*
- *PHP key categories are practicality, power, possibility, and price.*
- *PHP is a embedded language that is commonly embedded on HTML*
- *PHP is a server-side scripting language that runs its application on a Web server.*
- *Much of the PHP syntax were derive from C programming Language.*
- *You can integrate database easily in PHP.*
- *Standard PHP script are enclosed by <?php and ?> tags.*
- *Each statement in PHP must be terminated with a semi-colon(;).*
- *All variable in PHP must have a dollar (\$) character before its identifier.*



Summary (Continue)

- *PHP variables are not explicitly declared.*
- *You can use echo, print, or printf function for outputting strings to internet browser.*
- *You can enclosed string using a pair of single quote or pair of double quote.*
- *String inside a pair of single quote interpreted as literal except for the single quote itself.*
- *String inside a pair of double quote interpreted as literal with some exception like using dollar character.*
- *To explicitly display the character on the internet browser use backslash (\) character like \\$, \', \” and etc.*
- *Datatypes in PHP can be categorized as scalar or compound datatypes.*
- *You can explicitly change the type of a variable in PHP.*
- *PHP support some predefined type function that can be used in manipulating type variables.*



Q&A SESSION



Applications Development and Emerging Technologies



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

MODULE 2

PHP Operators and Control Structures



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Objectives

- To understand the different types of operators that are available on PHP.
- To know what is operator precedence and operator associativity in PHP.
- To ensure that programs expression are logically correct by using the correct referencing values of its precedence.
- To use escape sequence properly in the program.
- To know the different approach of control structures.
- To know the fundamentals syntax for conditional and looping structures.
- To properly use the compound expression using the logical operators.
- To know the rules of break, continue, and goto statements.
- To see some alternative enclosure for control structures.



Operators

An operator is a symbol that specifies a particular action in an expression

Operator Precedence, Associativity, and Purpose

Operator	Associativity	Purpose
new	NA	Object instantiation
()	NA	Expression subgrouping
[]	Right	Index enclosure
! ~ ++ --	Right	Boolean NOT, bitwise NOT, increment, decrement
@	Right	Error suppression
/ * %	Left	Division, multiplication, modulus
+ - .	Left	Addition, subtraction, concatenation
<< >>	Left	Shift left, shift right (bitwise)
< <= > >=	NA	Less than, less than or equal to, greater than, greater than or equal to

PHP Operators and Control Structures

Operator Precedence, Associativity, and Purpose

Operator	Associativity	Purpose
& ^	Left	Bitwise AND, bitwise XOR, bitwise OR
&&	Left	Boolean AND, Boolean OR
?:	Right	Ternary operator
= += *= /= .= %=&= = ^= <<= >>=	Right	Assignment operators
AND XOR OR	Left	Boolean AND, Boolean XOR, Boolean OR
,	Left	Expression separation; example: \$days = array(1=>"Monday", 2=>"Tuesday")

Beginning PHP and MySQL 3rd Edition by: W. Jason Gilmore pp.124



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

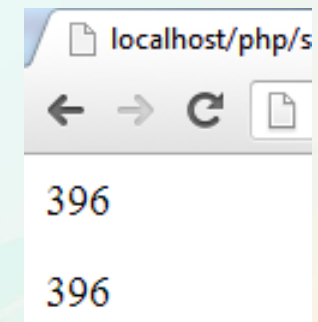
PHP Operators and Control Structures

Operator Precedence

is a characteristic of operators that determines the order in which they evaluate the operands surrounding them.

```
<?php
    $price = 450;
    $discount = 12;
    $value = $price - $price * $discount / 100;
    echo "<p>$value pesos</p>";
    //is the same as
    $value = ($price - (($price * $discount) / 100));
    echo "<p>$value pesos</p>";
?>
```

Output:



localhost/php/s

396

396



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

PHP Operators and Control Structures

Operator Associativity

is a characteristic of an operator specifies how operations of the same precedence are evaluated as they are executed.

Example:

```
<?php
$a = 600;
$b = 30;
$c = 5;
$x = $a / $b * $c;
echo "<p>$x</p>";
//is also same as
$x = (($a / $b) * $c); //left to right
echo "<p>$x</p>";

$x = $y = $z = $c;
echo "<p>$x</p>";
//is also same as
($x = ($y = ($z = $c))); //right to lefts
echo "<p>$x</p>";
?>
```

Output:

100

100

5

5



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

PHP Operators and Control Structures

Arithmetic Operator

Example	Label	Outcome
<code>\$a + \$b</code>	Addition	Sum of \$a and \$b
<code>\$a - \$b</code>	Subtraction	Difference of \$a and \$b
<code>\$a * \$b</code>	Multiplication	Product of \$a and \$b
<code>\$a / \$b</code>	Division	Quotient of \$a and \$b
<code>\$a % \$b</code>	Modulus	Remainder of \$a divided by \$b

Beginning PHP and MySQL 3rd Edition by: W. Jason Gilmore pp.124

Assignment Operator

Example	Label	Outcome
<code>\$a = 5</code>	Assignment	\$a equals 5
<code>\$a += 5</code>	Addition-assignment	\$a equals \$a plus 5
<code>\$a *= 5</code>	Multiplication-assignment	\$a equals \$a multiplied by 5
<code>\$a /= 5</code>	Division-assignment	\$a equals \$a divided by 5
<code>\$a .= 5</code>	Concatenation-assignment	\$a equals \$a concatenated with 5

Beginning PHP and MySQL 3rd Edition by: W. Jason Gilmore pp.126



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

PHP Operators and Control Structures

Increment and Decrement Operator

Example	Label	Outcome
++\$a, \$a++	Increment	Increment \$a by 1
--\$a, \$a--	Decrement	Decrement \$a by 1

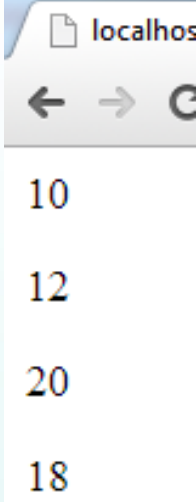
Beginning PHP and MySQL 3rd Edition by: W. Jason Gilmore pp.127

Example:

```
<?php
    $x = 10;
    //post increment
    echo "<p>".$x++."</p>"; //10
    //pre increment
    echo "<p>".++$x."</p>"; //12

    $y = 20;
    //post decrement
    echo "<p>".$y--."</p>"; //20
    //pre decrement
    echo "<p>".--$y."</p>"; //18
?>
```

Output:



10

12

20

18



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

PHP Operators and Control Structures

Comparison Operator

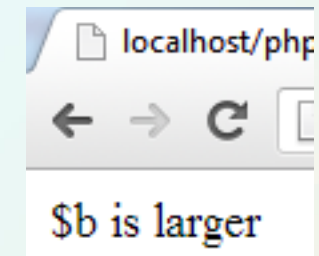
Example	Label	Outcome
<code>\$a < \$b</code>	Less than	True if \$a is less than \$b
<code>\$a > \$b</code>	Greater than	True if \$a is greater than \$b
<code>\$a <= \$b</code>	Less than or equal to	True if \$a is less than or equal to \$b
<code>\$a >= \$b</code>	Greater than or equal to	True if \$a is greater than or equal to \$b
<code>(\$a == 12) ? 5 : -1</code>	Ternary	If \$a equals 12, return value is 5; otherwise, return value is -1

Beginning PHP and MySQL 3rd Edition by: W. Jason Gilmore pp.129

Example:

```
<?php
    $a = 10;
    $b = 20;
    $x = $a > $b ? "\$a is larger": "\$b is larger";
    echo $x;
?>
```

Output:



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

PHP Operators and Control Structures

Logical Operator

Example	Label	Outcome
<code>\$a && \$b</code>	AND	True if both \$a and \$b are true
<code>\$a AND \$b</code>	AND	True if both \$a and \$b are true
<code>\$a \$b</code>	OR	True if either \$a or \$b is true
<code>\$a OR \$b</code>	OR	True if either \$a or \$b is true
<code>!\$a</code>	NOT	True if \$a is not true
<code>NOT \$a</code>	NOT	True if \$a is not true
<code>\$a XOR \$b</code>	Exclusive OR	True if only \$a or only \$b is true

Beginning PHP and MySQL 3rd Edition by: W. Jason Gilmore pp.128

Equality Operator

Example	Label	Outcome
<code>\$a == \$b</code>	Is equal to	True if \$a and \$b are equivalent
<code>\$a != \$b</code>	Is not equal to	True if \$a is not equal to \$b
<code>\$a === \$b</code>	Is identical to	True if \$a and \$b are equivalent and \$a and \$b have the same type

Beginning PHP and MySQL 3rd Edition by: W. Jason Gilmore pp.129



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

PHP Operators and Control Structures

Bitwise Operator

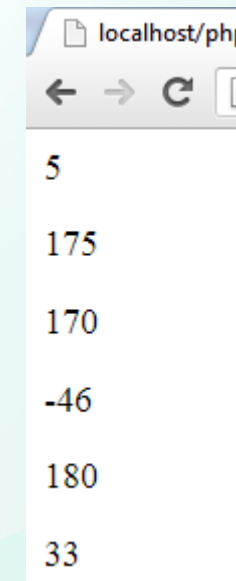
Example	Label	Outcome
<code>\$a & \$b</code>	AND	And together each bit contained in \$a and \$b
<code>\$a \$b</code>	OR	Or together each bit contained in \$a and \$b
<code>\$a ^ \$b</code>	XOR	Exclusive-or together each bit contained in \$a and \$b
<code>~ \$b</code>	NOT	Negate each bit in \$b
<code>\$a << \$b</code>	Shift left	\$a will receive the value of \$b shifted left two bits
<code>\$a >> \$b</code>	Shift right	\$a will receive the value of \$b shifted right two bits

Beginning PHP and MySQL 3rd Edition by: W. Jason Gilmore pp.130

Example:

```
<?php
$a = 135; $b = 45;
$c = $a & $b;
echo "<p>$c</p>";
$c = $a | $b;
echo "<p>$c</p>";
$c = $a ^ $b;
echo "<p>$c</p>";
$c = ~$b;
echo "<p>$c</p>";
$c = $b << 2;
echo "<p>$c</p>";
$c = $a >> 2;
echo "<p>$c</p>";
?>
```

Output:



```
localhost/php
5
175
170
-46
180
33
```



PHP Operators and Control Structures

Escape Sequences

Sequence	Description
\n	Newline character
\r	Carriage return
\t	Horizontal tab
\\	Backslash
\\$	Dollar sign
\"	Double quote
\[0-7]{1,3}	Octal notation
\x[0-9A-Fa-f]{1,2}	Hexadecimal notation

Beginning PHP and MySQL 3rd Edition by: W. Jason Gilmore pp.132



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

PHP Operators and Control Structures

PHP Control Structures: Conditional Statements

if statement

syntax:

```
if(expression) {  
    statement...  
}
```

else statement

syntax:

```
if(expression) {  
    statement...  
} else {  
    statement  
}
```

elseif statement

syntax:

```
if(expression) {  
    statement...  
}  
elseif(expression) {  
    statement...  
} else {  
    statement...  
}
```



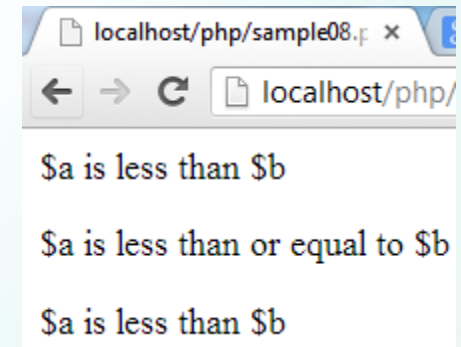
PHP Operators and Control Structures

PHP Control Structures: Conditional Statements

Example:

```
<?php
$a = 50; $b = 80;
//if statement
if($a < $b){
    echo "<p>\$a is less than \$b</p>";
}
//if...else statement
if($a > $b){
    echo "<p>\$a is greater than \$b</p>";
} else {
    echo "<p>\$a is less than or equal to \$b</p>";
}
//if...elseif...else
if($a > $b){
    echo "<p>\$a is greater than \$b</p>";
} elseif($a < $b) {
    echo "<p>\$a is less than \$b</p>";
} else {
    echo "<p>\$a is equal to \$b</p>";
}
?>
```

Output:



PHP Operators and Control Structures

PHP Control Structures: Conditional Statements Nested if...else

Example:

```
<?php
    $score = 86;
    if($score > 80){
        if($score < 91){
            echo "Your score is between 80 to 91";
        }else{
            echo "Your score is higher than 90";
        }
    }else{
        echo "Your score is lower than 81";
    }
?>
```

Output:

Your score is between 80 to 91



PHP Operators and Control Structures

PHP Control Structures: Conditional Statements Compound expression using logical operators

Example:

```
<?php
    $score = 86;
    if($score > 80 && $score<91){
        echo "Your score is between 80 to 91";
    } else if($score > 90){
        echo "Your score is higher than 90";
    } else{
        echo "Your score is lower than 81";
    }
?>
```

Output:

Your score is between 80 to 91



PHP Operators and Control Structures

PHP Control Structures: Conditional Statements

switch statement

syntax:

```
switch($category) {  
    case opt1:  
        statement...  
        break;  
    case opt2:  
        statement...  
        break;  
    case opt3:  
        statement...  
        break;  
    ...  
    default:  
        statement...  
}
```



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

PHP Operators and Control Structures

PHP Control Structures: Conditional Statements Switch Statement

Example:

```
<?php
    $month = 2;
    switch($month) {
        case 1:
            echo "January"; break;
        case 2:
            echo "February"; break;
        case 3:
            echo "March"; break;
        case 4:
            echo "and so on up to 12"; break;
        default:
            echo "Invalid"; break;
    }
?>
```

Output:

February



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

PHP Operators and Control Structures

PHP Control Structures: Looping Statements

while statement

syntax:

```
while (expression) {  
    statement...  
}
```

for statement

syntax:

```
for (expr1; expr2; expr3)  
{  
    statement...  
}
```

do ... while statement

syntax:

```
do {  
    statement...  
} while (expression) ;
```



PHP Operators and Control Structures

PHP Control Structures: Looping Statements

Example:

```
<?php
    $x = 10;
    while($x>0){
        echo $x--." ";
    }
    echo "<br/>";

    $y = 20;
    do{
        echo $y++." ";
    }while($y<21);
    echo "<br/>";

    $z=5;
    for($i=$z;$i>0;$i--){
        echo $i." ";
    }

?>
```

Output:

```
10 9 8 7 6 5 4 3 2 1
20
5 4 3 2 1
```

PHP Operators and Control Structures

PHP Control Structures: break, continue, and goto statement

break

break statement is placed within the code of a loop to cause the program to break out of the loop statement.

continue

continue statement causes execution of the current loop iteration to end and commence at the beginning of the next iteration.

goto ... label:

goto statement is used to jump to other section of the program to support labels.



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

PHP Operators and Control Structures

PHP Control Structures: break, continue, and goto statement

Example:

```
<?php
    $i=1;
    while(true){
        $i++;
        if($i<20){
            continue;
        }elseif($i>=20 && $i<30){
            echo $i."<br/>";
        }else{
            break;
        }
    }
    goto display;
    echo "This statement were skipped by goto";
display:
    echo "statement from display label";
?>
```

Output:

```
20
21
22
23
24
25
26
27
28
29
statement from display label
```



PHP Operators and Control Structures

PHP Control Structures: alternative enclosure syntax

Involves replacing the opening bracket with a colon(:) and replacing the closing bracket with endif;, endwhile;,endfor;, endswitch

Example:

```
<?php
    $a = 10; $b = 20; $c = 30;
    if($a<$b):
        echo "<p>\$a is less than \$b</p>";
    endif;
    for($i=1;$i<11;$i++):
        echo $i." ";
    endfor;
?>
```

Output:

\$a is less than \$b

1 2 3 4 5 6 7 8 9 10



Summary

- *Simplifying expression needs to follow a general precedence set by PHP scripts.*
- *An operator specifies a particular action in a given expression.*
- *Available PHP operators are arithmetic, conditional, assignment, logical and bitwise operator.*
- *Operator precedence characteristics determines the order in which they evaluate the operands surrounding them.*
- *Operator associativity characteristic specifies how operations of the same precedence are evaluated as they are executed.*
- *Post increment or decrement always execute the statement first before incrementing or decrementing the value of a given variable.*
- *Pre increment or decrement always increment or decrement its variable value before executing the statement.*
- *Escape sequence has its own special task in executing the string as an output.*



Summary (Continue)

- *Conditional statements are statement the will execute statement inside a block if a given condition is true*
- *Use compound expression to minimize the process of coding using nested if statement.*
- *break statement caused the program to break if used.*
- *continue statement end the current loop and continue to next iteration.*
- *goto statement used to jump to other section of the program.*
- *Alternative enclosure syntax is available for if, while, for, foreach, and switch control.*



Q&A SESSION



Applications Development and Emerging Technologies

MODULE 3

PHP Arrays and User Defined Functions



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Objectives

- *To know the different approach of using arrays in PHP.*
- *To be familiar with using print_r function and var_dump function for array inspection.*
- *To use PHP lazy function foreach to iterate through array elements.*
- *To implement one dimensional and multi-dimensional in the program.*
- *To know what is associative array in PHP.*
- *To use different predefined functions in manipulating array elements.*
- *To defined what is function and to use function keyword in declaring function in PHP.*
- *To create a user defined function using different approach.*



Objectives (continue)

- *To know use variables that is globally declared and locally declared.*
- *To create static variables that can be used in accumulating data every time the function calls.*

PHP Arrays



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

PHP Arrays and User Defined Functions

Array

is used to aggregate a series of similar items together, arranging and dereferencing them in some specific way.

- Each member of the array index references a corresponding value and can be a simple numerical reference to the value's position in the series, or it could have some direct correlation to the value.
- PHP array does not need to declare how many elements that the array variable have.
- Array index in PHP can be also called as array keys.
- Array can be used as ordinary array same as in C and C++ arrays.



PHP Arrays and User Defined Functions

Array: one dimensional

Example: Code 1

```
<?php
    $fruit[0] = "orange";
    $fruit[1] = "apple";
    $fruit[2] = "grapes";
    $fruit[3] = "banana";
    echo "<pre>";
    print_r($fruit);
    echo "</pre>";
?>
```

Example: Code 2

```
<?php
    $fruit[] = "orange";
    $fruit[] = "apple";
    $fruit[] = "grapes";
    $fruit[] = "banana";
    echo "<pre>";
    print_r($fruit);
    echo "</pre>";
?>
```

Example: Code 3

```
<?php
    $fruit = array("orange", "apple", "grapes", "banana");
    echo "<pre>";
    print_r($fruit);
    echo "</pre>";
?>
```

Output:

```
Array
(
    [0] => orange
    [1] => apple
    [2] => grapes
    [3] => banana
)
```



PHP Arrays and User Defined Functions

Array: Functions for visualizing arrays

`print_r()` function

short for print recursive. This takes an argument of any type and prints it out, which includes printing all its parts recursively.

`var_dump()` function

is same as the `print_r` function except that it prints additional information about the size and type of the values it discovers

`print_r()` and `var_dump()` functions are commonly used for debugging. The point of this of these functions is to help you visualize what's going on with compound data structures like arrays.



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

PHP Arrays and User Defined Functions

Array: Functions for visualizing arrays

Example: using `print_r()`

```
<?php
    $fruit = array(
        "orange",
        "apple",
        "grapes",
        "banana");
    echo "<pre>";
    print_r($fruit);
    echo "</pre>";
?>
```

Output:

```
Array
(
    [0] => orange
    [1] => apple
    [2] => grapes
    [3] => banana
)
```

Example: using `var_dump()`

```
<?php
    $fruit = array(
        "orange",
        "apple",
        "grapes",
        "banana");
    echo "<pre>";
    var_dump($fruit);
    echo "</pre>";
?>
```

Output:

```
array(4) {
    [0]=>
    string(6) "orange"
    [1]=>
    string(5) "apple"
    [2]=>
    string(6) "grapes"
    [3]=>
    string(6) "banana"
}
```

PHP Arrays and User Defined Functions

Array: Looping through array elements

`foreach()` function

is a statement used to iterate or loop through the element in an array. With each loop, a `foreach` statement moves to the next element in an array.

`foreach` statement

specify an array expression within a set of parenthesis following the `foreach` keyword.

Syntax

```
foreach($arr as $value){  
    //do something with $value variable  
}  
  
foreach($arr as $key => $value){  
    //do something with $key and/or $value variable  
}
```



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

PHP Arrays and User Defined Functions

Array: Looping through array elements

<code>\$arr</code>	The name of the array that you're walking through.
<code>\$key</code>	The name of the variable where you want to store the key. (optional)
<code>\$value</code>	The name of the variable where you want to store the value

Example:

```
<?php
    $fruit = array(
        "orange",
        "apple",
        "grapes",
        "banana",
        "dragon fruit");
    foreach($fruit as $value){
        echo "$value<br/>";
    }
?>
```

Output:

```
orange
apple
grapes
banana
dragon fruit
```



PHP Arrays and User Defined Functions

Array: Looping through array elements

Example:

```
<?php
    $person = array(
        'First Name' => 'Juan',
        'Last Name' => 'Dela Cruz',
        'Date of Birth' => 'April 3, 1986',
        'Gender' => 'Male'
    );

    foreach($person as $value){
        echo "$value <br/>";
    }
    echo "<p/>";
    foreach($person as $key => $value){
        echo "$key: $value <br/>";
    }
?>
```

Output:

Juan
Dela Cruz
April 3, 1986
Male

First Name: Juan
Last Name: Dela Cruz
Date of Birth: April 3, 1986
Gender: Male



PHP Arrays and User Defined Functions

Arrays: Multidimensional arrays

Example:

```
<?php
$person = array(
    'Leader' => array(
        'First Name' => 'Juan',
        'Last Name' => 'Dela Cruz',
        'Date of Birth' => 'April 3, 1986',
        'Gender' => 'Male' ),
    'Assistant' => array(
        'First Name' => 'Rica',
        'Last Name' => 'San Pedro',
        'Date of Birth' => 'January 8, 1980',
        'Gender' => 'Female' ),
    'Member' => array(
        'First Name' => 'Jose',
        'Last Name' => 'Dela Paz',
        'Date of Birth' => 'March 2 1985',
        'Gender' => 'Male' )
);

foreach($person as $pkey => $parr){
    echo "<h2>$pkey</h2>";
    foreach($parr as $key => $value){
        echo "$key: $value<br/>";
    }
}
```

Output:

Leader

First Name: Juan
Last Name: Dela Cruz
Date of Birth: April 3, 1986
Gender: Male

Assistant

First Name: Rica
Last Name: San Pedro
Date of Birth: January 8, 1980
Gender: Female

Member

First Name: Jose
Last Name: Dela Paz
Date of Birth: March 2 1985
Gender: Male

PHP Arrays and User Defined Functions

Arrays: Sorting

Function	Description
<code>sort(\$array)</code>	Sorts by value; assign new numbers as the keys
<code>rsort(\$array)</code>	Sorts by value in reverse order; assign new number as the keys
<code>asort(\$array)</code>	Sorts by value; keeps the same key
<code>arsort(\$array)</code>	Sorts by value in reverse order; keeps the same key
<code>ksort(\$array)</code>	Sorts by key
<code>krsort(\$array)</code>	Sorts by key in reverse order
<code>usort(\$array, functionname)</code>	Sorts by a function

PHP Arrays and User Defined Functions

Arrays: Sorting

Example:

```
<?php
    $fruit = array(
        "orange", "apple",
        "grapes", "banana",
        "guava", "blue berry");

    sort($fruit); //sort array in ascending
    echo "<pre>";
    print_r($fruit);
    echo "</pre>";

    rsort($fruit); //sort array in descending
    echo "<pre>";
    print_r($fruit);
    echo "</pre>";
?>
```

Output:

```
Array
(
    [0] => apple
    [1] => banana
    [2] => blue berry
    [3] => grapes
    [4] => guava
    [5] => orange
)

Array
(
    [0] => orange
    [1] => guava
    [2] => grapes
    [3] => blue berry
    [4] => banana
    [5] => apple
)
```



PHP Arrays and User Defined Functions

Arrays: Sorting

Example:

```
<?php
    $fruit = array(
        "orange", "apple",
        "grapes", "banana",
        "guava", "blue berry");

    asort($fruit);
    echo "<pre>";
    print_r($fruit);
    echo "</pre>";

    arsort($fruit);
    echo "<pre>";
    print_r($fruit);
    echo "</pre>";
?>
```

Output:

```
Array
(
    [1] => apple
    [3] => banana
    [5] => blue berry
    [2] => grapes
    [4] => guava
    [0] => orange
)

Array
(
    [0] => orange
    [4] => guava
    [2] => grapes
    [5] => blue berry
    [3] => banana
    [1] => apple
)
```



PHP Arrays and User Defined Functions

Arrays: Sorting

Example:

```
<?php
$fruit = array(
    '1010' => "orange",
    '1002' => "apple",
    '1023' => "grapes",
    '1006' => "banana",
    '1015' => "guava",
    '1009' => "blue berry");

ksort($fruit);
echo "<pre>";
print_r($fruit);
echo "</pre>";

krsort($fruit);
echo "<pre>";
print_r($fruit);
echo "</pre>";
```

?>

Output:

```
Array
(
    [1002] => apple
    [1006] => banana
    [1009] => blue berry
    [1010] => orange
    [1015] => guava
    [1023] => grapes
)
```

```
Array
(
    [1023] => grapes
    [1015] => guava
    [1010] => orange
    [1009] => blue berry
    [1006] => banana
    [1002] => apple
)
```



PHP User Defined Functions



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

PHP Arrays and User Defined Functions

Functions

is a group of PHP statements that performs a specific task. Functions are designed to allow you to reuse the same code in different locations.

User defined functions

functions that are provided by the user of the program.

Predefined functions

functions that are built-in into PHP to perform some standard operations



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

PHP Arrays and User Defined Functions

Functions: User defined functions

Syntax

```
function name(param) {  
    //code to be executed by the function  
}
```

Where

function - is the keyword used to declare a function

name - is the name of the function or function identifier

param - is the formal parameters of the function.

Parameter must follow the rule of naming identifier.

PHP Arrays and User Defined Functions

Functions: Function with no parameters

Example:

```
<?php
function redline() {
    ?>
    <hr style="
background-color:#f00;
height:5px;
    " />
<?php
}

function footer() {
    echo "PHP Test Page &copy 2013";
}

echo "First Text";
redline();
echo "Second Text";
redline();
footer();
?>
```

Output:

First Text

Second Text

PHP Test Page © 2013



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

PHP Arrays and User Defined Functions

Functions: Function with parameters

Example:

```
<?php
function hrline($height, $color){
    ?>
    <hr
        style="
            height:<?php echo $height; ?>;
            background-color:<?php echo $color ?>"
        />
    <?php
}

echo "First Text";
hrline("5px", "#00f");
echo "second Text";
hrline("10px", "#0f0");
?>
```

Output:

First Text



second Text



PHP Arrays and User Defined Functions

Functions: Function that returns a value

Example:

```
<?php
function qoutient($x, $y){
    return $x/$y;
}

function remark($grade){
    if($grade>=75){
        return true;
    } else {
        return false;
    }
}

printf("The qoutient is %.2f<br/>",qoutient(7,3));
if(remark(80)){
    echo "passed";
} else {
    echo "failed";
}

?>
```

Output:

The qoutient is 2.33
passed



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

PHP Arrays and User Defined Functions

Functions: Nested function

Example:

```
<?php
function displayPerson() {
    function name() {
        echo "Juan Dela Cruz";
    }
}
```

Output:

```
name();
```

Fatal error: Call to undefined function name() in C:\wamp\www\php\sample11.php on line 8

```
?>
```

Example:

```
<?php
function displayPerson() {
    function name() {
        echo "Juan Dela Cruz";
    }
}
displayPerson();
name();
```

```
?>
```

Output:

Juan Dela Cruz



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

PHP Arrays and User Defined Functions

Functions: Variable scope

Global Variables

is one that declared outside a function and is available to all parts of the program.

Local Variables

is declared inside a function and is only available within the function in which it is declared.

Static Variables

is used to retain the values calls to the same function.



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

PHP Arrays and User Defined Functions

Functions: using variables

Example:

```
<?php
    $str = "This is global variable";
    function f(){
        echo "function f() was called<br>";
        echo "$str<br>";
    }
    f();
?>
```

Output:

function f() was called

Example:

```
<?php
    $str = "This is global variable";
    function f(){
        global $str;
        echo "function f() was called<br>";
        echo "$str<br>";
    }
    f();
?>
```

Output:

function f() was called
This is global variable



PHP Arrays and User Defined Functions

Functions: using variables

Example:

```
<?php
function f(){
    $str = "This is local variable";
}
f();
echo "Displaying local variable: ";
echo $str;
?>
```

Output:

Displaying local variable:

Example:

```
<?php
function f(){
    global $str;
    $str = "This is local variable";
}
f();
echo "Displaying local variable: ";
echo $str;
?>
```

Output:

Displaying local variable: This is local variable



PHP Arrays and User Defined Functions

Functions: using variables

Example:

```
<?php
function counter($value){
    static $count = 0;
    $count+=$value;
    echo "The value of the counter is $count<br>";
}

counter(4);
counter(3);
counter(8);
counter(6);
?>
```

Output:

```
The value of the counter is 4
The value of the counter is 7
The value of the counter is 15
The value of the counter is 21
```



Summary

- *Array is used to aggregate a series of similar items together.*
- *Array index references a corresponding value.*
- *Array index can be simple numerical or have some direct correlation to the value.*
- *Array index is also known as Array Keys.*
- *print_r function is used to print the array structure.*
- *var_dump function is same as print_r function except it adds additional information about the data of each element.*
- *The foreach statement is use to iterate through the element in an array.*
- *Using foreach statement you can display both the keys and value of each element in the array.*
- *PHP provides functions for array manipulation such as sort(), rsort(), asort(), arsort(), ksort(), krsort(), and usort() functions.*

Summary (continue)

- *sort(), asort(), and ksort() functions are used to sort elements in the array in ascending order.*
- *rsort(), arsort(), and krsort() functions are used to sort elements in the array in descending order.*
- *sort() and rsort() does not maintain its index reference for each values.*
- *asort(), ksort(), arsort(), and krsort() maintains its reference for each values.*
- *asort() and arsort() used to sort elements by values.*
- *ksort() and krsort() used to sort elements by keys.*
- *Functions is a group of PHP statements that performs a specific task.*
- *Functions can be user defined generally defined by the user of the program and predefined that are build in using libraries.*

Summary (continue)

- *You use functions in different ways. Function can only do something without passing values. You can pass values to a function and you can ask functions to return a value.*
- *function keyword is used in PHP to declare a function.*
- *A function that is declared inside a function is said to be hidden.*
- *To gain access to a variable that is outside from the function we use the global keyword.*
- *We use static keyword to declare a variable inside a function that will act as accumulator variable this will let the program remember the last value of the variable that was used.*



Q&A SESSION



Applications Development and Emerging Technologies

MODULE 4

PHP Predefined Functions



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Objectives

- *To know how to include separate PHP code in the main page for code enhancement.*
- *To be familiar with the use of common predefined function such as define, include, and require.*
- *To use different available mathematical function for manipulating numbers.*
- *To use number_format() function for number readability purpose.*
- *To apply different function for Array Manipulation using the function unset(), explode(), and implode().*
- *To manipulate string data using PHP string manipulation function.*
- *To manipulate date with format using date function and date format available.*



PHP Predefined Functions

Using Constant

`define()` functions

used to declare constants. A constant can only be assigned a scalar value, like a string or a number. A constant's value cannot be changed.

Syntax:

```
define('NAME', 'value');
```

Example:

```
<?php
define('MAX_VALUE',10);
for($i=5;$i<MAX_VALUE;$i++){
    echo $i." ";
}
//MAX_VALUE = 20; is invalids
?>
```

Output:

5 6 7 8 9



PHP Predefined Functions

Including Files

You can separate your PHP file and embed it to your html by using PHP include functions.

Include functions	Description
<code>include</code>	Includes and evaluates the specified file. Generate a warning on failure message if file not found.
<code>require</code>	Performs the same way as the include function. Generate a fatal error message if file not found stopping the script at that point.
<code>include_once</code>	Same as include function except it includes the file only once.
<code>require_once</code>	Same as require function except it includes the file only once.



PHP Predefined Functions

Including Files

Most of the developers used include functions for their header and footer. Also some use this to write their database connection and so on.

You may write the file with an extension name of .inc rather than .php to serve as a fragment of your program code.

In some scripts, a file might be included more than once, causing function redefinitions, variable reassignments, and other possible problems.

Syntax:

```
include("filename.inc");  
include_once("filename.inc");  
require("filename.inc");  
require_once("filename.inc");
```



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

PHP Predefined Functions

Including Files: `include()` function (file not found)

Example:

```
<html>
  <head>
    <title></title>
    <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
  </head>
  <body>
    <?php include("somefile.inc") ?>
    <h4>Content</h4>
    The quick brown fox jumps over the lazy dog near the bank of the rivers
  </body>
</html>
```

Output:

Warning: include(somefile.inc) [[function.include](#)]: failed to open stream: No such file or

Warning: include() [[function.include](#)]: Failed opening 'somefile.inc' for inclusion (include

Content

The quick brown fox jumps over the lazy dog near the bank of the river



PHP Predefined Functions

Including Files: `require()` function (file not found)

Example:

```
<html>
  <head>
    <title></title>
    <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
  </head>
  <body>
    <?php require("somefile.inc") ?>
    <h4>Content</h4>
    The quick brown fox jumps over the lazy dog near the bank of the rivers
  </body>
</html>
```

Output:

Warning: require(somefile.inc) [[function.require](#)]: failed to open stream: No such file or

Fatal error: require() [[function.require](#)]: Failed opening required 'somefile.inc' (include_1



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

PHP Predefined Functions

Including Files: `include()` function (file exists)

Example:

```
<html>
  <head>
    <title></title>
    <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
  </head>
  <body>
    <?php include("header.inc") ?>
    <h4>Content</h4>
    The quick brown fox jumps over the lazy dog near the bank of the rivers
  </body>
</html>
```

header.inc

```
<h1> Some header text</h2>
```

Output:

Some header text

Content

The quick brown fox jumps over the lazy dog near the bank of the rivers

Same output using `require` function

Technology Driven by Innovation



FEU ALABANG



FEU DILIMAN



FEU TECH

PHP Predefined Functions

Mathematical Function:

rand() function

- used to generate random integers
- syntax

```
int rand(void)
int rand(int $min, int $max)
```

ceil() function

- returns the next highest integer by rounding the value upwards
- syntax

```
float ceil(float $value)
```

floor() function

- returns the next lowest integer by rounding the value downwards
- syntax

```
float floor(float $value)
```



PHP Predefined Functions

Mathematical Function:

min() function

- Return the smallest value
- syntax

```
mixed min(array $values)
```

```
mixed min(mixed $values1, mixed $values2[,mixed $...])
```

max() function

- Return the highest value
- syntax

```
mixed max(array $values)
```

```
mixed max(mixed $values1, mixed $values2[,mixed $...])
```



PHP Predefined Functions

Mathematical Function:

`rand()` , `ceil()` , `floor()` , `min()` , `max()`

Example:

```
<?php
for($i=0;$i<5;$i++){
    $xgen[] = rand();
}
echo "x values: ";
foreach($xgen as $num){
    echo $num." ";
}
echo "<br />";
echo "min: ".min($xgen)." max: ".max($xgen);
echo "<br />";
for($i=0;$i<5;$i++){
    $ygen[] = rand(5,10);
}
echo "y values: ";
foreach($ygen as $num){
    echo $num." ";
}
echo "<br />";
echo "min: ".min($ygen)." max: ".max($ygen);
echo "<br />";
echo "ceil(3.01): ".ceil(3.01)."<br/>";
echo "floor(3.91): ".floor(3.91)."<br/>";
?>
```

Output:

x values: 16516 24685 29858 13619 24816
min: 13619 max: 29858
y values: 6 7 10 6 7
min: 6 max: 10
ceil(3.01): 4
floor(3.91): 3



PHP Predefined Functions

Mathematical Function:

`number_format()` function

- Format a number with grouped thousand
- syntax

```
string number_format (  
    float $number  
    [, int $decimals = 0 ] )
```

```
string number_format (  
    float $number ,  
    int $decimals = 0 ,  
    string $dec_point = '.' ,  
    string $thousands_sep = ',' )
```

PHP Predefined Functions

Mathematical Function: `number_format` function

Example:

```
<?php
$price = 217795.75;
echo number_format($price)."<br>";
echo number_format($price,2,'.','')."<br>";
echo number_format($price,2,'.','')."<br>";
?>
```

Output:

```
217,796
217795.75
217,795.75
```



PHP Predefined Functions

Function for Array Manipulation:

`unset` function

- destroys the specified variable
- syntax:

```
void unset ( mixed $var [, mixed $... ] )
```

`explode` function

- split a string by string
- syntax:

```
array explode ( string $delimiter ,  
               string $string [, int $limit ] )
```

`implode` function

- join array elements to form a string
- syntax:

```
string implode ( string $glue , array $pieces )  
string implode ( array $pieces )
```



PHP Predefined Functions

Function for Array Manipulation:

`unset()` , `explode()` , `implode()`

Example: `<?php`

```
$fruitArr = array('orange','apple','grapes','mango','banana');
foreach($fruitArr as $fruit) echo $fruit." ";

$fruitStr = implode(", ", $fruitArr);
echo "<p/>Fruit string(implode): $fruitStr";

unset($fruitArr);
echo "<p/>Fruit list (unset)<pre>"; print_r($fruitArr); echo "</pre>";

$fruitArr = explode(", ", $fruitStr);
foreach($fruitArr as $fruit) echo $fruit." ";

?>
```

Output: orange apple grapes mango banana

Fruit string(implode): orange,apple,grapes,mango,banana

Fruit list (unset)

orange apple grapes mango banana

PHP Predefined Functions

Function for String Manipulation:

`strlen` function

- return the value length of a string
- syntax:

```
int strlen (string $string)
```

`strpos` function

- find the position of the first occurrence of a substring in a given string
- syntax:

```
int strpos ( string $haystack ,  
            mixed $needle [, int $offset = 0 ] )
```

`strrev` function

- reverse a given string
- syntax:

```
string strrev ( string $string )
```



PHP Predefined Functions

Function for String Manipulation:

`strtolower` function

- converts string to lowercase
- syntax:

```
string strtolower ( string $str )
```

`strtoupper` function

- converts string to uppercase
- syntax:

```
string strtoupper ( string $str )
```

`substr` function

- returns part of a given string
- syntax:

```
string substr ( string $string ,  
                int $start [, int $length ] )
```



PHP Predefined Functions

Function for String Manipulation:

`strlen()`, `strpos()`, `strrev()`, `strtolower()`,
`strtoupper()`, `substr()`

Example:

```
<?php
```

```
$str = "The quick brown fox jumps over the lazy dog";  
printf("Length of string: %s <br/>", strlen($str));  
printf("fox string is at position: %d <br/>", strpos($str,'fox'));  
printf("String reverse: %s <br/>", strrev($str));  
$abcStr = "AbCdEfghIjKL";  
printf("String in lowercase: %s <br/>", strtolower($abcStr));  
printf("String in uppercase: %s <br/>", strtoupper($abcStr));  
printf("Sub String \"fox jumps\": %s <br/>", substr($str, 16, 9));
```

```
?>
```

Output: Length of string: 43
fox string is at position: 16
String reverse: god yzal eht revo spmuj xof nworb kciuq ehT
String in lowercase: abcdefghijkl
String in uppercase: ABCDEFGHIJKL
Sub String "fox jumps": fox jumps



PHP Predefined Functions

Function for String Manipulation:

`ucfirst()` function

- Make a string's first character uppercase
- syntax:

```
string ucfirst( string $str )
```

`ucwords()` function

- converts string to uppercase
- syntax:

```
string ucwords ( string $str )
```

`trim()` function

- stripped white spaces or other characters from the beginning and end of a string.
- syntax:

```
string trim ( string $str [, string $charlist ] )
```



PHP Predefined Functions

Function for String Manipulation:

`ltrim()` function

- strip white spaces or other characters from the beginning of a string.

- **syntax:**

```
string ltrim ( string $str [, string $charlist ] )
```

`rtrim()` function

- strip white spaces or other characters from the end of a string.

- **syntax:**

```
string rtrim ( string $str [, string $charlist ] )
```

`strip_tags()` function

- strip HTML and PHP tags from a string.

- **syntax:**

```
string strip_tags ( string $str [, string  
$allowable_tags ] )
```



PHP Predefined Functions

Function for String Manipulation:

`ucfirst()`, `ucwords()`, `trim()`, `ltrim()`, `rtrim()`, `strip_tags()`

```
<?php
```

```
$str = "the quick bron fox jumps...";  
echo ucfirst($str). "<br/>";  
echo ucwords($str). "<br/>";  
$str = "  sample string  ";  
echo "<pre>"; var_dump($str); echo "</pre>";  
$str = ltrim($str);  
echo "<pre>"; var_dump($str); echo "</pre>";  
$str = rtrim($str);  
echo "<pre>"; var_dump($str); echo "</pre>";  
$str = "  sample string  ";  
echo "<pre>"; var_dump($str); echo "</pre>";  
$str = trim($str);  
echo "<pre>"; var_dump($str); echo "</pre>";  
$str = "<p>paragraph</p><hr/><a href='#'>link</a>";  
$strA = strip_tags($str);  
echo "<pre>"; var_dump($strA); echo "</pre>";  
$strB = strip_tags($str, '<a><p>');  
echo $strB;
```

```
?>
```

Example:

Output:

The quick bron fox jumps...

The Quick Bron Fox Jumps...

string(19) " sample string "

string(16) "sample string "

string(13) "sample string"

string(19) " sample string "

string(13) "sample string"

string(13) "paragraphlink"

paragraph

[link](#)

PHP Predefined Functions

Function for String Manipulation:

`ucfirst()`, `ucwords()`, `trim()`, `ltrim()`, `rtrim()`, `strip_tags()`

```
<?php
```

```
$str = "the quick bron fox jumps...";  
echo ucfirst($str). "<br/>";  
echo ucwords($str). "<br/>";  
$str = "  sample string  ";  
echo "<pre>"; var_dump($str); echo "</pre>";  
$str = ltrim($str);  
echo "<pre>"; var_dump($str); echo "</pre>";  
$str = rtrim($str);  
echo "<pre>"; var_dump($str); echo "</pre>";  
$str = "  sample string  ";  
echo "<pre>"; var_dump($str); echo "</pre>";  
$str = trim($str);  
echo "<pre>"; var_dump($str); echo "</pre>";  
$str = "<p>paragraph</p><hr/><a href='#'>link</a>";  
$strA = strip_tags($str);  
echo "<pre>"; var_dump($strA); echo "</pre>";  
$strB = strip_tags($str, '<a><p>');  
echo $strB;
```

```
?>
```

Example:

Output:

The quick bron fox jumps...

The Quick Bron Fox Jumps...

string(19) " sample string "

string(16) "sample string "

string(13) "sample string"

string(19) " sample string "

string(13) "sample string"

string(13) "paragraphlink"

paragraph

[link](#)

PHP Predefined Functions

Date Manipulation: date () Function

- used to format a local time or date
- returns a string formatted according to the given format string.
- syntax:

```
string date ( string $format [, int $timestamp = time() ] )
```

Format character	Description	Example returned values
DAY		
d	Day of the month, 2 digits with leading zeros	01 to 31
D	A textual representation of a day, three letters	Mon through Sun
j	Day of the month without leading zeros	1 to 31
l	A full textual representation of the day of the week	<i>Sunday</i> through <i>Saturday</i>
N	ISO-8601 numeric representation of the day of the week (added in PHP 5.1.0)	1 (for Monday) through 7 (for Sunday)

PHP Predefined Functions

Date Manipulation: `date ()` Function

Format character	Description	Example returned values
S	English ordinal suffix for the day of the month, 2 characters	<i>st, nd, rd</i> or <i>th</i> . Works well with <i>j</i>
w	Numeric representation of the day of the week	0 (for Sunday) through 6 (for Saturday)
z	The day of the year (starting from 0)	0 through 365
WEEK		
W	ISO-8601 week number of year, weeks starting on Monday	Example: 42 (the 42nd week in the year)



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

PHP Predefined Functions

Date Manipulation: date () Function

Format character	Description	Example returned values
MONTH		
F	A full textual representation of a month, such as January or March	<i>January through December</i>
m	Numeric representation of a month, with leading zeros	<i>01 through 12</i>
M	A short textual representation of a month, three letters	<i>Jan through Dec</i>
n	Numeric representation of a month, without leading zeros	<i>1 through 12</i>
t	Number of days in the given month	<i>28 through 31</i>

PHP Predefined Functions

Date Manipulation: `date ()` Function

Format character	Description	Example returned values
YEAR		
L	Whether it's a leap year	1 if it is a leap year, 0 otherwise.
o	ISO-8601 year number. This has the same value as Y, except that if the ISO week number (W) belongs to the previous or next year, that year is used instead.	Examples: 1999 or 2003
Y	A full numeric representation of a year, 4 digits	Examples: 1999 or 2003
y	A two digit representation of a year	Examples: 99 or 03

PHP Predefined Functions

Date Manipulation: `date ()` Function

Format character	Description	Example returned values
TIME		
a	Lowercase Ante meridiem and Post meridiem	<i>am</i> or <i>pm</i>
A	Uppercase Ante meridiem and Post meridiem	<i>AM</i> or <i>PM</i>
B	Swatch Internet time	<i>000</i> through <i>999</i>
g	12-hour format of an hour without leading zeros	<i>1</i> through <i>12</i>
G	24-hour format of an hour without leading zeros	<i>0</i> through <i>23</i>
h	12-hour format of an hour with leading zeros	<i>01</i> through <i>12</i>
H	24-hour format of an hour with leading zeros	<i>00</i> through <i>23</i>
i	Minutes with leading zeros	<i>00</i> to <i>59</i>
s	Seconds, with leading zeros	<i>00</i> through <i>59</i>
u	Microseconds	Example: <i>654321</i>

PHP Predefined Functions

Date Manipulation: `date ()` Function

Format character	Description	Example returned values
TIMEZONE		
e	Timezone identifier	Examples: <i>UTC, GMT, Atlantic/Azores</i>
I	Whether or not the date is in daylight saving time	<i>1</i> if Daylight Saving Time, <i>0</i> otherwise.
O	Difference to Greenwich time (GMT) in hours	Example: <i>+0200</i>
P	Difference to Greenwich time (GMT) with colon between hours and minutes	Example: <i>+02:00</i>
T	Timezone abbreviation	Examples: <i>EST, MDT</i> ...
Z	Timezone offset in seconds. The offset for timezones west of UTC is always negative, and for those east of UTC is always positive.	<i>-43200</i> through <i>50400</i>

PHP Predefined Functions

Date Manipulation: `date ()` Function

Format character	Description	Example returned values
FULL DATE/TIME		
c	ISO 8601 date	2004-02-12T15:19:21+00:00
r	» RFC 2822 formatted date	Example: <i>Thu, 21 Dec 2000 16:01:07 +0200</i>
U	Seconds since the Unix Epoch (January 1 1970 00:00:00 GMT)	See also <code>time()</code>

<http://php.net/docs.php>



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

PHP Predefined Functions

Date Manipulation: date () Function

Example:

```
<?php
    echo date("l");
    echo "<br/>";
    echo date('l jS \of F Y h:i:s A');
?>
```

Output:

```
Wednesday
Wednesday 6th of February 2013 08:35:22 AM
```



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

PHP Predefined Functions

Date Manipulation: `mktime()` function

- get the unix timestamp (January 1, 1970) for a given date. (with strict notice)
- same as `time()` function (without strict notice)
- syntax:

```
int mktime ([ int $hour = date("H")  
            [, int $minute = date("i")  
            [, int $second = date("s")  
            [, int $month = date("n")  
            [, int $day = date("j")  
            [, int $year = date("Y")  
            [, int $is_dst = -1 ]]]]] )
```



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

PHP Predefined Functions

Date Manipulation: `strtotime()` function

- parse any English textual datetime description into a Unix timestamp
- syntax:

```
int strtotime ( string $time  
    [, int $now = time() ] )
```

Q&A SESSION

